BGLP – Big Gym Landing Page

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	BGLP — Big Gym Landing Page
Gerente de Projeto	Enzo Henrique Viana
Gerente/Responsável de GCS-DevOps	Enzo Henrique Viana (Desenvolvedor Líder)
Fábrica de Software / Squad	Squad enxuta: Desenvolvedor Líder + par de revisão
Par de revisão	Nome do colega — @usuario-github
Repositório principal	https://github.com/viaaanx/biggym-landing-page

Histórico de Revisões

Data	Versão	Descrição	Autor
26/08/2026	0.1	Estrutura inicial do documento a partir do template RUP 7.0 adaptado; definição preliminar do escopo (repositório big-gym-landing-page) e preenchimento da tabela de identificação do projeto.	Enzo Henrique Viana
02/09/2026	0.2	Elaboração da Seção 1 (Introdução): finalidade, escopo detalhado dos itens de configuração, tabela de definições/acrônimos, referências e visão geral contrastando GCS manual vs. automatizada.	Enzo Henrique Viana
09/09/2026	0.3	Elaboração da Seção 2 (Cultura DevOps e Papéis), formalizando o modelo de responsabilidade compartilhada "you build it, you run it" e a reatribuição do papel do CCM a um quality gate automatizado, adequado à squad individual.	Enzo Henrique Viana
16/09/2026	0.4	Elaboração da Seção 3 (Ferramentas, Ambientes e Infraestrutura): matriz de ferramentas do pipeline, definição dos três ambientes (local, preview efêmero, produção) e estratégia de branching/versionamento (GitHub Flow, Conventional Commits, SemVer).	Enzo Henrique Viana
22/09/2026	1.0	Revisão geral, consolidação e formatação final dos Itens 1 a 3 do PGC para entrega, com enfoque em práticas DevOps (CI/CD) adaptadas ao repositório big-gym-landing-page.	Enzo Henrique Viana
14/10/2026	1.1	Adaptação do plano à implantação prática no repositório, conforme as atividades da disciplina (ONGConecta como referência): par de revisão com aprovação obrigatória, ambientes staging e produção com aprovação manual no GitHub Pages, merge somente por squash com versão calculada a partir do título do PR, testes automatizados com cobertura mínima e revisão das Seções 4 a 14. O projeto passa a ser versionado no repositório biggym-landing-page, que preserva o histórico do repositório anterior.	Enzo Henrique Viana

Sumário

Sumário 3
1. Introdução 5
1.1 Finalidade 5
1.2 Escopo 5
1.3 Definições, Acrônimos e Abreviações 6
1.4 Referências 6
1.5 Visão Geral 7
2. Cultura DevOps e Papéis na Gerência de Configuração 7
3. Ferramentas, Ambientes e Infraestrutura 8
3.1 Ferramentas do Pipeline 8
3.2 Ambientes e Infraestrutura 9
3.3 Estratégia de Branching e Versionamento 10
4. Identificação da Configuração 11
4.1 Itens de Configuração 11
4.2 Convenção de Nomenclatura 12
4.3 Estrutura do Repositório 12
5. Integração Contínua (CI) 13
5.1 Etapas do Pipeline de CI 13
5.2 Política de Pull Request e Revisão de Código 13
5.3 Critérios de Qualidade (Quality Gate) 14
6. Entrega e Implantação Contínua (CD) 15
6.1 Etapas do Pipeline de CD 15
6.2 Estratégias de Implantação 15
6.3 Rollback e Recuperação 15
7. Infraestrutura como Código (IaC) 16
7.1 Versionamento de Infraestrutura 16
7.2 Ambientes Efêmeros 16
8. Controle de Configuração e Mudança 16
8.1 Processo de Solicitação de Mudança 16
8.2 Comitê de Controle de Mudança (CCM) 16
9. Baselines e Releases 17
9.1 Geração Automática de Baseline 17
9.2 Notas de Versão (Release Notes) 17
10. Estimativa do Status de Configuração 17
10.1 Monitoramento e Observabilidade 17
10.2 Métricas DORA 17
10.3 Relatórios e Auditorias 18
11. Marcos 18

12. Treinamento e Recursos..... 18
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros..... 19
14. Quadro-Resumo do Pipeline CI/CD..... 19

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) aplicadas ao projeto BGLP — Big Gym Landing Page, repositório público hospedado em https://github.com/viaaanx/biggym-landing-page, cujo produto é uma landing page institucional (HTML5, CSS3 e JavaScript vanilla) desenvolvida para uma academia real e publicada via GitHub Pages. O plano tem enfoque em práticas de DevOps — Integração Contínua (CI), Entrega Contínua (CD) e automação de pipelines — para substituir controles documentais manuais por evidências automatizadas de rastreabilidade: histórico de commits, execuções de pipeline, aprovações registradas, tags de versão e artefatos publicados.

O projeto é conduzido por um único desenvolvedor, que atua como Desenvolvedor Líder e acumula os papéis de Product Owner, Engenheiro DevOps, Desenvolvedor, QA e responsável por segurança. Como o GitHub não permite que o autor aprove o próprio Pull Request, a revisão de código é feita por um par de revisão: um colega da disciplina com acesso de escrita ao repositório, que revisa e aprova as mudanças antes do merge (e cujo projeto é revisado em contrapartida). Os controles automatizados do pipeline complementam essa revisão humana, sem substituí-la.

1.1 Finalidade

Este PGC tem por finalidade estabelecer, de forma formal e vinculante para todas as contribuições ao repositório BGLP, um padrão único de identificação, versionamento, controle de mudanças e auditoria dos itens de configuração do produto. Especificamente, o plano busca:

- Padronizar a forma como o código-fonte (HTML, CSS, JavaScript), os artefatos de configuração de pipeline e a documentação técnica são identificados, nomeados e versionados;
- Garantir rastreabilidade fim-a-fim entre uma mudança solicitada (issue), o código que a implementa (commit/branch), a revisão realizada (Pull Request) e a versão publicada em produção (tag/release);
- Eliminar controles manuais e propensos a falha humana como deploys realizados via upload manual de arquivos, anotações de versão em planilhas paralelas ou aprovações verbais substituindo-os por gates automatizados de qualidade dentro do pipeline de CI/CD;
- Assegurar que qualquer estado do site publicado em produção seja reproduzível a partir do histórico versionado do repositório, sem dependência de conhecimento tácito do autor.

1.2 Escopo

Este PGC é destinado ao desenvolvedor responsável pelo projeto BGLP (Enzo Henrique Viana) e ao seu par de revisão, abrangendo integralmente o repositório https://github.com/viaaanx/biggym-landing-page e todos os artefatos nele versionados, incluindo:

- Código-fonte da aplicação: index.html (estrutura semântica), style.css (Flexbox/Grid) e script.js (Lightbox de galeria, carrossel de depoimentos em JavaScript vanilla, animações via Intersection Observer, contadores animados);
- Artefatos de SEO e dados estruturados: sitemap.xml, robots.txt e o bloco JSON-LD (schema.org/LocalBusiness) embutido no index.html, utilizados para indexação e rich snippets no Google;
- Artefatos estáticos versionados no diretório /assets (imagens da academia, ícones e mídias utilizadas na galeria e na seção de instrutores);
- Ferramentas de qualidade e testes: package.json e package-lock.json (versões exatas das ferramentas), arquivos de configuração de lint, testes automatizados em /tests e scripts de build em /scripts;
- Pipeline e colaboração como código: workflows do GitHub Actions (.github/workflows/), modelos de Pull Request e de Issue, CODEOWNERS e a configuração do Dependabot;

- Documentação técnica: README.md e este próprio PGC, versionado em /docs;
- A infraestrutura de hospedagem, limitada ao serviço gerenciado GitHub Pages, publicado a partir da branch gh-pages (produção na raiz e staging em /staging). Não há servidores, containers ou bancos de dados sob responsabilidade da squad, o que reduz a superfície de GCS a código-fonte, pipeline e conteúdo estático versionado.

Ficam fora do escopo deste plano: sistemas de terceiros consumidos apenas por link externo (ex.: WhatsApp Business, Google Maps incorporado via iframe), cuja configuração não é versionada neste repositório, e qualquer eventual back-end futuro, que exigiria revisão e extensão deste PGC.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega Contínua (Continuous Delivery): toda mudança mesclada chega a staging automaticamente e à produção mediante aprovação
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiu um estado comprovado de estabilidade; aqui, cada tag vX.Y.Z publicada em produção
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
GitHub Flow	Estratégia de branching com uma única branch principal (main) sempre implantável e branches curtas integradas por Pull Request
PR	Pull Request: proposta de mudança revisada e validada antes do merge na main
Par de revisão	Colega com permissão de escrita que revisa e aprova os PRs do projeto
Squash merge	Merge que junta todos os commits do PR em um único commit na main, cuja mensagem é o título do PR
Quality gate	Conjunto de critérios objetivos que o pipeline exige para liberar o merge
Environment	Ambiente de implantação do GitHub (staging, production) com regras de proteção, como aprovação obrigatória
Smoke test	Verificação rápida, após o deploy, de que a versão nova está no ar
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH), ex.: 1.4.2
SCA	Análise de Composição de Software: verificação de vulnerabilidades em dependências
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei 13.709/2018)
DORA Metrics	Métricas de desempenho DevOps (Deployment Frequency, Lead Time for Changes, Change Failure Rate, MTTR)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS — material da disciplina de Engenharia de Software, Católica SC.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado
- Conventional Commits, especificação v1.0.0 — conventionalcommits.org.

- Semantic Versioning, especificação 2.0.0 — semver.org.
- Documentação oficial do GitHub: GitHub Actions, GitHub Pages e Branch Protection Rules — docs.github.com.
- Google web.dev, documentação do Lighthouse e do Lighthouse CI; documentação do Vitest, ESLint, Stylelint e HTMLHint.
- Repositório de referência ONGConecta (github.com/lzcamargo/ongconecta) e roteiros das atividades práticas em docs/aulas, que orientam a implantação deste plano.
- DORA — Accelerate: State of DevOps Reports (Google Cloud), como referência para as métricas de desempenho descritas neste plano.
- Repositório-fonte do projeto: https://github.com/viaaanx/biggym-landing-page e ambiente de produção: https://viaaanx.github.io/biggym-landing-page/.

1.5 Visão Geral

Em um modelo tradicional e burocrático de GCS, a publicação de uma alteração na landing page dependeria de um ciclo predominantemente manual: o desenvolvedor editaria os arquivos localmente, validaria visualmente no navegador, copiaria (ou faria upload via FTP/painel de hospedagem) os arquivos alterados para o servidor de produção, e registraria a mudança quando registrada em uma planilha de controle de versões apartada do código. Nesse cenário, não há garantia de que o que está em produção corresponde a um commit identificável, a validação de qualidade é subjetiva (visual, ad hoc) e o rollback depende de o desenvolvedor ter guardado manualmente uma cópia da versão anterior.

Neste plano, a GCS é parte constante e automatizada da esteira de entrega. Cada Pull Request aberto contra a branch main dispara um pipeline de CI (GitHub Actions) que executa lint de HTML, CSS e JavaScript, testes automatizados com cobertura mínima, análise de dependências e uma auditoria Lighthouse; um segundo workflow confere se o título do PR segue Conventional Commits. O merge só é liberado com o pipeline verde e a aprovação do par de revisão, regras impostas pela proteção da branch. Após o merge, o pipeline de CD calcula a próxima versão, gera o build uma única vez e o publica automaticamente em staging; a promoção do mesmo artefato à produção aguarda a aprovação do Desenvolvedor Líder. Cada publicação em produção deixa evidências permanentes — log do workflow, registro da aprovação, tag SemVer e Release com notas —, dispensando controles documentais paralelos.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS do projeto BGLP adota o princípio DevOps de responsabilidade compartilhada entre desenvolvimento e operação, "you build it, you run it": quem escreve o código também responde por fazê-lo chegar à produção com qualidade e rastreabilidade. Como o projeto tem um único desenvolvedor, a maioria dos papéis é exercida pela mesma pessoa (Enzo Henrique Viana), mas cada um é descrito separadamente porque delimita responsabilidades distintas. A exceção é a revisão de código, que exige outra pessoa: ela cabe ao par de revisão.

O Comitê de Controle de Mudanças (CCM) é formado pelo Desenvolvedor Líder e pelo par de revisão. Nas mudanças de rotina, sua função é cumprida pelo próprio fluxo de Pull Request: quality gate automatizado (Seção 5.3) mais aprovação do par. O CCM só é acionado formalmente nas mudanças de alto risco listadas na Seção 8.2.

Papel	Responsabilidades
Product Owner (Enzo)	Define e prioriza as mudanças de conteúdo e funcionalidade da landing page (novas seções, textos, imagens, campanhas), registra-as como Issues e valida o resultado em staging antes de aprovar a produção.
Desenvolvedor Líder / Engenheiro DevOps (Enzo)	Mantém os workflows do GitHub Actions, a estratégia de branching e versionamento (Seção 3.3), a proteção da branch main e os Environments; é o aprovador obrigatório do deploy em produção e o dono do código nas áreas definidas no CODEOWNERS.
Desenvolvedor (Enzo)	Implementa as mudanças de HTML/CSS/JS em branches curtas, abre os Pull Requests com título em Conventional Commits e checklist preenchido, e corrige as falhas apontadas pelo pipeline ou pela revisão.
Par de revisão (colega da disciplina)	Revisa os Pull Requests com base no checklist do template, testa o build gerado pelo CI quando a mudança é visual, comenta e aprova ou pede alterações. Sem a sua aprovação, o merge é bloqueado.
QA / Automação de Testes (Enzo)	Mantém os testes em /tests e as verificações do pipeline (lint, cobertura, Lighthouse), e valida a responsividade em celular e desktop antes de pedir revisão.
DevSecOps (Enzo)	Garante que segredos nunca sejam versionados, acompanha os alertas e PRs do Dependabot, mantém o npm audit no CI e assegura que o site seja servido apenas via HTTPS.
Comitê de Controle de Mudanças (Enzo + par)	Analisa e registra a decisão sobre mudanças de alto risco (Seção 8.2) por meio do modelo de Issue "Mudança de alto risco".

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

A matriz abaixo relaciona as ferramentas adotadas no pipeline do projeto BGLP. Todas são gratuitas ou incluídas no plano gratuito do GitHub para repositórios públicos, condição necessária para usar aprovação em Environments.

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento do código-fonte, dos artefatos estáticos, das configurações e do pipeline como código.
Repositório e colaboração	GitHub (repositório público), Issues, Pull Requests, CODEOWNERS	Histórico completo, modelos de PR e Issue, revisão por par e regras de proteção da main.
Orquestração de CI/CD	GitHub Actions	Workflows ci.yml, pr-titulo.yml, cd.yml, rollback.yml e monitoramento.yml.
Ferramentas e dependências	Node.js 22 + npm (package-lock.json)	Instalação reprodutível das ferramentas com npm ci. O site em si não depende de Node.
Linting de HTML	HTMLHint (.htmlhintrc)	Sintaxe, atributos obrigatórios (alt, lang, title), IDs únicos e pares de tags no index.html.
Linting de CSS	Stylelint (stylelint-config-recommended)	Erros e redundâncias no style.css: propriedades duplicadas ou inválidas, unidades em zero, valores redundantes.
Linting de JavaScript	ESLint 9 (eslint.config.js)	Erros estáticos no script.js, nos testes e nos scripts:

Categoria	Ferramenta(s) adotada(s)	Finalidade
		variáveis não usadas ou não declaradas, comparações inseguras.
Testes automatizados	Vitest + jsdom + cobertura v8	Comportamento do script.js (menu, lightbox, carrossel, contadores) e integridade do index.html (arquivos e links).
Auditoria de qualidade web	Lighthouse CI (lighthouserc.json)	Quality gate de Performance, Acessibilidade, SEO e Boas Práticas sobre o build de cada PR.
Validação do título do PR	Workflow pr-titulo.yml	Garante o padrão Conventional Commits no título, que vira o commit na main e define a versão.
Versionamento e releases	scripts/proxima-versao.mjs + GitHub Releases	Cálculo da próxima versão SemVer e das notas de release a partir dos commits.
Análise de dependências (SCA)	npm audit + GitHub Dependabot	npm audit no CI e PRs semanais de atualização para pacotes npm e GitHub Actions.
Gestão de segredos	GitHub Actions Secrets	Armazenamento de tokens; hoje só o GITHUB_TOKEN automático é usado.
Ambientes e aprovação	GitHub Environments (staging, production)	Restringe deploys à main e exige aprovação do Desenvolvedor Líder antes da produção.
Hospedagem / Deploy	GitHub Pages (branch gh-pages) + peaceiris/actions-gh-pages	Produção em https://viaaanx.github.io/biggym-landing-page/ e staging em /staging, com HTTPS e CDN do GitHub.
Smoke test e monitoramento	curl em scripts/aguardar-deploy.sh e workflow agendado	Confirma que a versão nova está no ar após cada deploy e verifica a produção periodicamente.

3.2 Ambientes e Infraestrutura

Por se tratar de um site inteiramente estático, a infraestrutura se resume a quatro ambientes lógicos derivados do mesmo repositório. O build é gerado uma única vez por versão e o mesmo artefato passa por staging e produção, o que garante paridade entre os dois.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento Local	Execução do site na máquina do desenvolvedor (npm start, em http://localhost:8080), com lint e testes rodando localmente antes do push.	Manual: git push da branch e abertura do PR.
Revisão de PR	Build do PR publicado como artefato site-pr-<n> na aba Actions, que o revisor baixa e abre localmente (npm start -- dist).	Cada push em um PR aberto contra a main.
Staging	https://viaaanx.github.io/biggym-landing-page/staging/ — Environment "staging", restrito à branch protegida.	Automático a cada merge na main, após o build.
Produção	https://viaaanx.github.io/biggym-landing-page/ — Environment "production", acessado pelos visitantes da academia.	Aprovação manual do Desenvolvedor Líder após validar staging.

Staging e produção convivem na branch gh-pages, em pastas distintas: o deploy de produção preserva a pasta /staging e vice-versa. O GitHub Pages não gera URLs de preview por Pull Request, por isso a revisão de PR usa o artefato de build (Seção 7.2). Os Environments aceitam deploys apenas a partir de branches protegidas, e a URL canônica declarada no index.html faz os buscadores tratarem a cópia de staging como duplicata da página oficial.

3.3 Estratégia de Branching e Versionamento

Adota-se o GitHub Flow: uma branch principal sempre implantável e branches de vida curta, integradas exclusivamente por Pull Request. A configuração do repositório impõe as regras abaixo; nenhuma delas depende de disciplina individual.

- main — tronco único, sempre implantável. Protegida: exige Pull Request com 1 aprovação do par de revisão, checks de CI obrigatórios, histórico linear e não admite exceções, nem para administradores. Push direto é recusado.
- feature/<issue>-<descrição-curta> — nova seção ou funcionalidade, idealmente concluída em 1 a 2 dias. Ex.: feature/12-plano-trimestral.
- fix/<issue>-<descrição-curta> — correção de defeito visual, de responsividade ou de comportamento. Ex.: fix/15-lightbox-teclado.
- hotfix/<descrição-curta> — correção urgente de algo já em produção (ex.: link do WhatsApp quebrado). Passa pelo mesmo Pull Request, revisão e pipeline; o que muda é a prioridade, e o registro como mudança de alto risco (Seção 8.2).
- Merge somente por squash, com o título e a descrição do PR como mensagem do commit; a branch é apagada automaticamente após o merge.

Como todo PR vira um único commit na main, é o título do Pull Request que precisa seguir Conventional Commits, no formato <tipo>(<escopo>): <descrição>; o workflow pr-titulo.yml bloqueia títulos fora do padrão. Os commits intermediários da branch também devem seguir a convenção, como boa prática. Os tipos usados no projeto são:

Tipo	Uso no contexto da landing page	Exemplo real
feat	Nova funcionalidade ou seção visível ao visitante. Gera versão MINOR.	feat(planos): adiciona plano trimestral
fix	Correção de defeito de comportamento, layout ou responsividade. Gera PATCH.	fix(lightbox): corrige navegação por teclado na galeria
perf	Melhoria de desempenho sem mudar o comportamento. Gera PATCH.	perf(hero): converte imagem do hero para WebP
style	Ajuste puramente visual que não corrige defeito nem adiciona funcionalidade.	style(footer): ajusta espaçamento entre ícones sociais
docs	Documentação (README, PGC, comentários).	docs(readme): atualiza instruções de execução local
test	Criação ou ajuste de testes automatizados.	test(carrossel): cobre botão de voltar
ci	Workflows do GitHub Actions e configuração do pipeline.	ci: adiciona auditoria Lighthouse ao workflow de CI
chore	Manutenção de dependências e configurações de ferramentas.	chore(deps): atualiza stylelint
refactor	Reestruturação interna sem alteração de comportamento visível.	refactor(script): extrai lógica do carrossel para função dedicada

O versionamento segue o Versionamento Semântico (SemVer — MAJOR.MINOR.PATCH) e é calculado pelo pipeline a partir dos títulos mesclados desde a última tag: feat incrementa MINOR, fix e perf incrementam PATCH, e um "!" após o tipo (ex.: feat!: nova identidade visual) ou a expressão BREAKING CHANGE na descrição incrementa MAJOR, reservado a redesigns estruturais. Os demais tipos não geram release. A primeira release é a 1.0.0, e a tag só é criada depois que a versão chega à produção (Seção 9), de modo que toda tag corresponde a um build validado em staging e aprovado.

4. Identificação da Configuração

4.1 Itens de Configuração

Os seguintes artefatos são itens de configuração formais do projeto BGLP, versionados no mesmo repositório Git e sujeitos ao mesmo fluxo de Pull Request:

- Código-fonte da aplicação: index.html, style.css e script.js;
- Artefatos estáticos: diretório /assets (imagens e ícones), robots.txt e sitemap.xml;
- Dados estruturados e metadados de SEO: bloco JSON-LD (schema.org/LocalBusiness), URL canônica e meta tags Open Graph/Twitter Cards no index.html;
- Dependências das ferramentas: package.json e package-lock.json, com versões exatas instaladas sempre por npm ci, e .nvmrc fixando o Node.js 22;
- Configuração de qualidade: .htmlhintrc, .stylelintrc.json, eslint.config.js, vitest.config.js e lighthouserc.json — regras versionadas junto ao código, aplicadas igualmente na máquina do desenvolvedor e no pipeline;

- Testes automatizados: diretório /tests;
- Scripts: scripts/build.mjs (monta o dist/ publicado), scripts/servidor.mjs (servidor local), scripts/proxima-versao.mjs (versão e notas) e scripts/aguardar-deploy.sh (smoke test);
- Pipeline como código: .github/workflows/ (ci.yml, pr-titulo.yml, cd.yml, rollback.yml, monitoramento.yml) e .github/dependabot.yml;
- Modelos de colaboração: .github/pull_request_template.md, .github/ISSUE_TEMPLATE/ (funcionalidade, defeito, mudança de alto risco) e .github/CODEOWNERS;
- Higiene do repositório: .gitignore, .gitattributes (finais de linha LF) e .env.example, que documenta variáveis de ambiente sem valores reais. Segredos nunca são versionados;
- Documentação: README.md (como rodar o site e os testes) e este PGC, em /docs;
- Scripts de migração de banco de dados: não aplicável — o projeto não possui persistência de dados nem back-end.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<issue>-<descrição-curta>	feature/12-plano-trimestral
Branch de correção	fix/<issue>-<descrição-curta>	fix/15-lightbox-teclado
Branch de correção emergencial	hotfix/<descrição-curta>	hotfix/link-whatsapp
Título do PR (= commit na main)	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(planos): adiciona plano trimestral
Tag de release	Versionamento Semântico vX.Y.Z	v1.2.0
Release no GitHub	Mesmo nome da tag, com notas geradas	v1.2.0
Workflow de pipeline	<finalidade>.yml em .github/workflows/	ci.yml, cd.yml
Artefato de build	site-pr-<n> (revisão) e site-dist (release)	site-pr-7
Commit de deploy na gh-pages	deploy(<ambiente>): <versão>	deploy(producao): v1.2.0
Environment	Nome em minúsculas	staging, production

4.3 Estrutura do Repositório

O repositório mantém o site na raiz, como já era servido pelo GitHub Pages, e separa em diretórios próprios tudo o que não é publicado:

- /index.html, /style.css e /script.js — a página, a folha de estilos e a lógica de interface;
- /assets, /robots.txt e /sitemap.xml — imagens e artefatos de SEO;
- /tests — testes automatizados (Vitest);
- /scripts — build, servidor local, cálculo de versão e smoke test;
- /.github — workflows, Dependabot, CODEOWNERS e modelos de PR e Issue;
- /docs — este PGC;

- Raiz — README.md, package.json, package-lock.json e arquivos de configuração das ferramentas.

O diretório dist/ é gerado pelo comando npm run build e nunca é versionado. Ele recebe apenas os arquivos listados explicitamente em scripts/build.mjs (index.html, style.css, script.js, assets, robots.txt e sitemap.xml), de modo que configurações, testes e workflows jamais chegam ao GitHub Pages.

5. Integração Contínua (CI)

Todo Pull Request aberto contra a branch main dispara automaticamente o workflow ci.yml. Nenhuma mudança é integrada à main sem que ele termine com sucesso, regra imposta pela proteção da branch, que exige os checks ci-ok e titulo-pr. O job ci-ok agrega o resultado dos demais: assim, um único check obrigatório cobre todo o pipeline.

5.1 Etapas do Pipeline de CI

1. Checkout do código da branch do Pull Request;
2. Configuração do Node.js 22 com cache do npm;
3. Instalação das ferramentas com npm ci, exatamente nas versões do package-lock.json;
4. Lint: npm run lint executa HTMLHint, Stylelint e ESLint;
5. Testes: npm run test:coverage executa o Vitest e falha se a cobertura ficar abaixo do mínimo da Seção 5.3;
6. Análise de dependências: npm audit --omit=dev --audit-level=critical;
7. Build: npm run build gera o dist/, publicado como artefato site-pr-<n> para a revisão do par;
8. Auditoria Lighthouse CI sobre o dist/, com os limites da Seção 5.3 e link para o relatório no resumo da execução;
9. Em paralelo, o workflow pr-titulo.yml valida o título do PR, inclusive quando ele é editado.

5.2 Política de Pull Request e Revisão de Código

- Toda mudança, de qualquer tamanho, é proposta via Pull Request; push direto na main é recusado pelo GitHub.
- Todo PR precisa de 1 aprovação do par de revisão. O GitHub não permite aprovar o próprio PR, e aprovações anteriores são descartadas quando entram commits novos.
- O autor preenche o checklist do template de PR (lint e testes locais, conferência em celular e desktop, links de WhatsApp e mapa, imagens otimizadas, ausência de segredos) e vincula a Issue com "Closes #<n>".
- O CODEOWNERS solicita automaticamente o Desenvolvedor Líder em mudanças de pipeline, scripts, dependências e configuração de qualidade.
- PRs devem ser pequenos e tratar de um único assunto. O par responde em até 3 dias úteis.

- As regras abaixo são configuradas no repositório e não admitem exceção, nem para o administrador:

Configuração (GitHub)	Valor
Require a pull request before merging	Ativado, com 1 aprovação obrigatória
Dismiss stale pull request approvals	Ativado
Require status checks to pass	ci-ok e titulo-pr (a partir da implantação do CI)
Require linear history	Ativado
Do not allow bypassing the above settings	Ativado
Tipos de merge permitidos	Somente squash, com título e descrição do PR como mensagem
Automatically delete head branches	Ativado
Colaboradores	Par de revisão com permissão Write
Environment staging	Deploy apenas a partir de branches protegidas
Environment production	Required reviewers: Desenvolvedor Líder; deploy apenas a partir de branches protegidas

5.3 Critérios de Qualidade (Quality Gate)

Critério	Limite mínimo exigido para o merge
Lint de HTML (HTMLHint)	Zero erros no index.html.
Lint de CSS (Stylelint)	Zero erros no style.css, segundo o stylelint-config-recommended e as regras do .stylelintrc.json.
Lint de JavaScript (ESLint)	Zero erros (regras recomendadas + eqeqeq, no-var e prefer-const).
Testes automatizados (Vitest)	Todos os testes passando.
Cobertura de testes	Mínimo de 60% de linhas, declarações e funções e 50% de ramos no script.js.
Dependências (npm audit)	Nenhuma vulnerabilidade crítica em dependências de produção.
Título do PR	Conventional Commits válido.
Lighthouse — Performance	Mínimo de 85/100 em modo mobile simulado (mediana de 3 execuções).
Lighthouse — Acessibilidade	Mínimo de 90/100, dado o caráter público do site.
Lighthouse — SEO	Mínimo de 90/100, já que a indexação orgânica é objetivo central.
Lighthouse — Boas Práticas	Mínimo de 90/100 (HTTPS, ausência de erros no console).

Medição de referência feita na adoção deste plano (outubro de 2026): Acessibilidade 100, SEO 100, Boas Práticas 96 e Performance 72. A Performance abaixo do limite vem da imagem do hero, um PNG de 2 MB; convertida para WebP (87 KB), a Performance sobe para 94. Essa conversão é a primeira mudança submetida ao fluxo, antes da ativação do Lighthouse como check obrigatório. A cobertura medida dos testes é de 92% das linhas do script.js.

6. Entrega e Implantação Contínua (CD)

Após o merge na main, o workflow cd.yml promove a mudança até a produção sem nenhuma cópia manual de arquivos. O build é feito uma única vez e o mesmo artefato vai para staging e, depois da aprovação, para a produção. Dois deploys nunca rodam ao mesmo tempo: o segundo aguarda o primeiro terminar.

6.1 Etapas do Pipeline de CD

1. Disparo automático por push na main (mudanças apenas em docs/ ou em arquivos .md não disparam deploy);
2. Cálculo da próxima versão e das notas de release a partir dos títulos mesclados desde a última tag (scripts/proxima-versao.mjs);
3. Build único com npm run build, gravando a versão na página (<meta name="app-version">) e guardando o dist/ como artefato por 30 dias;
4. Deploy em staging: publicação do artefato na pasta /staging da branch gh-pages;
5. Smoke test em staging: o script aguardar-deploy.sh consulta a URL até encontrar a versão nova publicada;
6. Gate de release: o job de produção fica pausado em "Waiting for review" até a aprovação do Desenvolvedor Líder no Environment production;
7. Deploy em produção: publicação do mesmo artefato na raiz da gh-pages, preservando a pasta /staging;
8. Smoke test em produção, com o mesmo critério do staging;
9. Criação da baseline: tag vX.Y.Z anotada e Release com as notas, somente quando há feat, fix ou perf desde a última tag.

6.2 Estratégias de Implantação

Por se tratar de um site estático hospedado em uma plataforma totalmente gerenciada (GitHub Pages), não há múltiplas instâncias, balanceador de carga ou orquestração de containers sob controle do projeto — o que torna estratégias como rolling update, blue-green ou canary não aplicáveis na infraestrutura atual, já que a própria CDN do GitHub Pages substitui a nova versão de forma atômica. Ainda assim, os conceitos abaixo são documentados como referência caso o projeto evolua para uma infraestrutura própria (ex.: hospedagem em nuvem com múltiplas instâncias):

Estratégia	Descrição
Rolling update	Não aplicável no GitHub Pages, que substitui a versão de forma atômica; relevante apenas com infraestrutura própria.
Blue-Green	Aproximado pelo par staging/produção: o mesmo artefato é validado em uma URL isolada e só então promovido, e a versão anterior continua disponível para rollback.
Canary Release	Não aplicável — o GitHub Pages não direciona fração do tráfego para uma versão específica.
Feature Flags	Aplicável a funcionalidades de maior risco (ex.: novo formulário): a seção pode ser publicada oculta e ativada depois por uma mudança pequena.

6.3 Rollback e Recuperação

Toda implantação é reversível sem editar arquivos em produção. Há dois caminhos, ambos sujeitos à mesma aprovação do Environment production: (a) o workflow rollback.yml, executado manualmente na aba Actions

com a tag desejada (ex.: v1.2.0), reconstrói o site a partir daquela tag e o publica em produção; (b) a reexecução do job de produção de uma execução anterior do CD, que reaproveita o artefato guardado por 30 dias. O rollback volta apenas a produção; em seguida, a main é corrigida por um PR com git revert ou com a correção definitiva. Uma falha no smoke test marca a execução como falha e notifica o responsável, que decide pelo rollback; a reversão automática fica registrada como evolução futura.

7. Infraestrutura como Código (IaC)

O projeto BGLP não provisiona servidores, redes ou bancos de dados — a hospedagem é delegada ao GitHub Pages —, o que dispensa ferramentas como Terraform ou Ansible. O princípio de infraestrutura como código se aplica ao que existe de configuração de plataforma: os workflows do GitHub Actions e o dependabot.yml, versionados no repositório, e as regras de proteção e Environments, documentados na Seção 5.2.

7.1 Versionamento de Infraestrutura

- Os workflows (.github/workflows/*.yml) e o dependabot.yml são versionados com a aplicação, alterados apenas via Pull Request e revisados obrigatoriamente pelo Desenvolvedor Líder (CODEOWNERS);
- Essas alterações usam o tipo ci da Seção 3.3, o que as torna rastreáveis no histórico;
- A proteção da main e os Environments são configurados pela interface do GitHub, mas seus valores estão registrados na tabela da Seção 5.2. Qualquer mudança neles é de alto risco (Seção 8.2) e exige atualização deste PGC.

7.2 Ambientes Efêmeros

O GitHub Pages publica um único site por repositório e não gera URLs de preview por Pull Request. O ambiente efêmero de cada PR é, portanto, o artefato site-pr-<n> gerado pelo CI: o revisor o baixa na aba Actions e o abre localmente com npm start -- dist. O artefato expira automaticamente em 7 dias, sem acumular recursos. Um preview por URL (Netlify ou Vercel) fica registrado como evolução possível, mas exigiria conta e token em outro serviço.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina seguem um fluxo padronizado e rastreável de ponta a ponta: abertura de uma Issue pelo modelo adequado → branch feature/ ou fix/ a partir da main → commits → Pull Request com título em Conventional Commits, checklist preenchido e "Closes #<n>" → CI (ci-ok e titulo-pr) → revisão e aprovação do par → squash merge → deploy automático em staging → validação → aprovação da produção → tag e Release. O próprio fluxo é o registro formal da mudança, sem formulário ou planilha paralela.

8.2 Comitê de Controle de Mudança (CCM)

O CCM é formado pelo Desenvolvedor Líder e pelo par de revisão. Nas mudanças de rotina, seu papel é cumprido pelo quality gate e pela aprovação do PR. O acionamento formal do CCM — uma análise registrada antes de prosseguir — é exigido nas seguintes situações:

- Alteração dos workflows de CI/CD, dos Environments ou das regras de proteção da main (mudança na forma como as mudanças são controladas);
- Alteração de domínio, de HTTPS ou da configuração do GitHub Pages;

- Hotfix em produção: segue o fluxo normal de PR e revisão, mas com prioridade, e é registrado para análise posterior;
- Inclusão de serviço ou biblioteca de terceiros que colete dados de visitantes, dada a implicação na LGPD.

Nesses casos, abre-se uma Issue pelo modelo "Mudança de alto risco (CCM)", que exige a descrição da mudança e o plano de rollback. A decisão do comitê (aprovada, aprovada com ressalvas ou rejeitada, com justificativa) é registrada nos comentários da Issue, e o PR correspondente a referencia.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada implantação aprovada em produção gera automaticamente uma baseline: uma tag Git anotada vX.Y.Z (Seção 3.3) apontando para o commit da main que originou a publicação, criada pelo próprio pipeline depois do smoke test. Somente versões que passaram por staging e foram aprovadas recebem tag. A tag permite reconstruir o site publicado em qualquer momento do histórico — é a partir dela que o workflow de rollback trabalha — sem backups manuais.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas pelo pipeline a partir dos títulos mesclados desde a última tag, agrupadas em Funcionalidades (feat), Correções (fix e perf) e Outros, e publicadas como uma Release do GitHub vinculada à tag. Como o título de cada PR é validado no CI, as notas saem consistentes sem redação manual de changelog.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

A produção é monitorada de forma leve, compatível com um site estático: o workflow agendado monitoramento.yml verifica a disponibilidade da URL de produção a cada 6 horas e executa uma auditoria Lighthouse semanal, para detectar degradação de desempenho, acessibilidade ou SEO causada por fatores externos (ex.: fontes ou mapa de terceiros). Uma falha no workflow gera notificação por e-mail e é registrada como Issue de defeito.

10.2 Métricas DORA

Métrica	Descrição	Aplicação ao projeto BGLP
Deployment Frequency	Frequência com que o código é implantado em produção.	Número de deploys aprovados no Environment production por semana, no histórico de Deployments do repositório.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.	Tempo entre a abertura do PR e a conclusão do deploy em produção, ambos registrados no GitHub.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.	Proporção de deploys em produção seguidos de rollback ou hotfix no período.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.	Tempo entre a detecção (smoke test, monitoramento ou Issue de defeito) e a conclusão do rollback ou da correção.

10.3 Relatórios e Auditorias

O histórico do GitHub — execuções do Actions (CI, CD, rollback e monitoramento), Deployments com o registro de quem aprovou cada produção, Releases, commits, Pull Requests com as revisões do par e Issues — constitui o insumo de auditoria de GCS do projeto. Os rollbacks deixam um resumo na própria execução. Quando necessário, por exemplo como evidência para a disciplina, esse histórico é exportado pela interface ou pela API do GitHub.

11. Marcos

Marco	Descrição
M1 — Repositório preparado (Aula 1)	Itens de configuração básicos (README, .gitignore, .gitattributes, lockfile, .env.example), modelos de PR e Issue, CODEOWNERS, par de revisão como colaborador, merge somente por squash, Environments staging e production e proteção da main.
M2 — Integração Contínua (Aula 2)	Workflows ci.yml e pr-titulo.yml, checks ci-ok e titulo-pr obrigatórios na proteção da main e Lighthouse CI no quality gate.
M3 — Entrega Contínua (Aula 3)	Workflows cd.yml e rollback.yml, publicação em staging e produção pela branch gh-pages, gate de aprovação e primeira release automatizada (v1.0.0).
M4 — Governança (Aula 4)	Dependabot para npm e GitHub Actions, uso do modelo de mudança de alto risco e rastreabilidade Issue → PR → Release.
M5 — Monitoramento contínuo	Workflow agendado de disponibilidade e Lighthouse em produção e acompanhamento das métricas DORA (Seção 10).

12. Treinamento e Recursos

O treinamento concentra-se na capacitação do Desenvolvedor Líder nas ferramentas adotadas e no alinhamento com o par de revisão sobre o processo de revisão:

- GitHub Actions: sintaxe de workflows, eventos (pull_request, push, schedule, workflow_dispatch), Environments com aprovação e controle de concorrência;
- Conventional Commits e Versionamento Semântico aplicados ao título do PR, de forma que versão e notas de release sejam calculadas corretamente;
- Ferramentas de lint (HTMLHint, Stylelint, ESLint), testes com Vitest e leitura dos relatórios do Lighthouse CI, para corrigir falhas do quality gate com autonomia;
- Revisão de código em par: uso do checklist do template, teste do artefato do PR, comentários objetivos e decisão entre aprovar ou pedir alterações;
- Recursos de apoio: documentação oficial do GitHub, do Lighthouse e do Vitest; repositório de referência ONGConecta (README, guia de onboarding, decisões de arquitetura e guia da rede restrita do laboratório); material da disciplina referenciado na Seção 1.4.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

O projeto BGLP não possui subcontratados ou fornecedores de desenvolvimento externos. Ainda assim, depende de serviços e pacotes de terceiros, tratados como itens de configuração sob controle:

- Serviços incorporados: Google Maps (iframe), Google Fonts e links para WhatsApp e Instagram. Não há configuração versionada além das URLs no index.html; os links de WhatsApp e o mapa são verificados pelos testes automatizados;
- Bibliotecas ou fontes carregadas via CDN devem ter a versão fixada na URL, nunca @latest, para que uma atualização do fornecedor não altere o site sem uma mudança no repositório;
- Pacotes npm: o site não tem dependências de produção; as ferramentas de lint, teste e build são dependências de desenvolvimento, com versões exatas no package-lock.json e instaladas sempre por npm ci;
- SCA: o npm audit roda no CI e bloqueia vulnerabilidades críticas em dependências de produção. Alertas em dependências de desenvolvimento, que não chegam ao site publicado, são acompanhados pelo Dependabot;
- O Dependabot abre PRs semanais de atualização dos pacotes npm (prefixo chore(deps)) e mensais das GitHub Actions (prefixo ci). Essas atualizações passam pelo mesmo CI e pela mesma revisão do par, nunca são aplicadas direto em produção.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Lint (HTMLHint, Stylelint, ESLint), testes com cobertura, npm audit, build e Lighthouse; validação do título do PR.	Abertura ou atualização de PR contra a main.
Revisão	Par de revisão analisa o PR e o artefato site-pr-<n> e aprova ou pede alterações.	Checks ci-ok e titulo-pr verdes.
Merge	Squash merge na main; o título do PR vira o commit e define a versão.	1 aprovação + CI verde.
CD — Staging	Cálculo da versão, build único, publicação em /staging e smoke test.	Push (merge) na main.
CD — Produção	Publicação do mesmo artefato na raiz e smoke test.	Aprovação do Desenvolvedor Líder no Environment production.
Baseline e Release	Tag vX.Y.Z e Release com notas geradas.	Deploy em produção concluído, havendo feat, fix ou perf.
Rollback	Reconstrução a partir de uma tag anterior e publicação em produção.	Execução manual do workflow rollback.yml, com aprovação.